2026-1 인공지능라이브러리 중간고사(20241519 조예성)

지도학습 종류 2개: 분류, 회귀

비지도학습 종류 2개: 군집화, 차원축소

강화학습 요소 5개: 에이전트, 환경, 상태, 행동, 보상

종속변수에 영향을 주는 변수: 독립변수
- 영향을 받는 변수: 종속변수

평균을 기준으로 얼마나 퍼져있는지: 분산
- (2, 4, 6)의 분산: 8/3

수학적 연산을 그래프로: 계산 그래프
- 연산은(노드)로 흐름은(엣지)로

미리 그래프 정의 후 실행: 정적 계산 그래프

코드 흐름에 따라 그래프 구조가 달라지고 디버깅 가능: 동적 계산 그래프

텐서는 (다차원 배열)이다

0차원 텐서: 스칼라
1차원 텐서: 벡터
2차원 텐서: 행렬
n차원 배열: 텐서

텐서의 차원이 같으면 (원소별)로 연산이 수행

차원이 다르면: 브로드캐스팅

GPU 연산은 기본적으로 동기적이다: O, **X**

텐서를 GPU로: tensor.to(device)

GPU 사용 가능 여부: torch.cuda.is_available()

동기화 코드: torch.cuda.synchronize()

신경망 기본 구성 요소: 뉴런

생물학적 뉴런에서 영감 받아 설계된 단위 연산자: 뉴런

이전 층 뉴런에서 출력되는 값: 입력

입력된 값의 중요도: 가중치

활성화 함수 전 값 조절 상수: 편향

선형 연산을 비선형을 변형: 활성화 함수

자동 미분 가능한 텐서 생성: torch.tensor(val, requires_grad = True)

텐서는 (1)개의 자료형만 가능하다

Start 부터 stop까지 n 개의 배열 생성: arrange

Start부터 stop까지 n개로 균등 분할: linspace

신경망에서 입력 데이터가 각 층을 통과하며 출력 생성하는 과정: 순전파

손실 최소화 위해 가중치와 편향 조정: 역전파

XOR 풀이: 두 값이 다르면 True, 같으면 False
- 선형 분리가 가능하다: O, **X**

회귀 주로 사용 손실 함수: MSE

분류 오차 계산법: CEE

경사하강법 성능 저하 문제 개선: 확률적 경사 하강법, 미니배치 경사하강법

기울기를 이용해 가중치를 어떻게 업데이트 할지: 옵티마이저_
- 가장 보편적인 옵티마이저: adam

PyTorch에서 신경망 클래스 제작 시 상속받아야 하는 것: nn.Module

정밀도 식: TP / (TP+FP)

재현율 식: TP / (TP+FN)

과적합 방지 기법: EarlyStopping, Dropout, Batch Normalization

파이토치에서 단계마다 가중치 초기화 해주는 코드: optimizer.zero_grad()

설정된 텐서의 모든 연산 기록 자동 저장: requires_grad = True

파라미터의 새로운 값 계산법: 이전 값에서 학습률과 해당 파라미터의 기울기 곱을 뺌
- 빼는 이유: 경사 하강법에서 경사를 내려가기 위하여
이미지의 밝깃값이나 기타 수치형 데이터를 일정 범위로 조정하는 작업: 정규화

이미지 기준 원본 데이터에 회전, 이동, 크기 조절 등 다양한 변환 적용해 학습: 데이터증강

데이터셋을 학습, 검증, 테스트 데이터로 나눔: 데이터 분할

한번의 여러 샘플 처리: 배치 처리

Epoch 마다 데이터 무작위로 섞기: 데이터 셔플

여러 프로세스를 동시에 불러오기: 멀티프로세싱

미리 정해진 여러 하이퍼파라미터값을 격자처럼 배열해 모두 시험: Grid Search

설정 범위 내 랜덤한 값 선택 후 시험: Random Search

파라미터와 모델 성능 간 관계를 확률 모델과 qcquisition function 통해 추정: Bayesian Optimization

CNN 장점: 이미지 2D 구조 유지, 효율성 향상, 부분적 불변성
- 핵심 요소: 합성곱, 활성화 함수, 풀링 레이어

커널 이동 거리: Stride
- 연산 후 출력값 크기 동일하게 유지 위한 것: Padding
  - 패딩 없는거: valid
  - 연산 후 크기 동일하게: same

가장 큰 값: Max Pooling
- 평균값: Average Pooling

전역 풀링: GAP

```
# 더 다양한 데이터 증강 기법 적용
transform_train_augmented = transforms.Compose([
  transforms.RandomCrop(32, padding=4), # 랜덤 크롭
  transforms.RandomHorizontalFlip(), # 수평 뒤집기
  transforms.RandomRotation(15), # 랜덤 회전(±15도)
  transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
  # 색상 변화
  transforms.ToTensor(), 0~1 로 나눠지기 전에 적용
  transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616))
])
```

합성곱 3x3 일관적 신경망: VGGNet
- MaxPooling 커널: 2x2
- Stride: 2

마이크로스프트 개발 네트워크: ResNet
- 사용 블록: 레지듀얼 블록
- 구조: 아이덴티티블록, 합성곱 블록
- 입력값을 출력에 바로: skip connection

모델 압축 기법: 가지치기, 필터 가지치기, 양자화

중요도가 낮은 개별 가중치 제거: 가지치기
CNN에서 중요도 낮은 필터 일부 제거: 필터 가지치기

모델 가중치와 활성화 값을 정밀도가 낮은 데이터 타입으로 변환: 양자화
- 장점: 크기감소, 속도 향상, 메모리 사용 감소

큰 모델 지식을 작은모델로 전달: 지식 증류
_ 교사모델의 (softmax 확률 분포)를 모방

CNN 주목 확인: CAM; Class Acvivation Map
- 사용 가능한 네트워크: GAP가 존제하는 NW

상관없이 적용 가능한 네트워크: GradCAM

시퀀스 데이터를 처리하기 위한 반복 구조 신경망: RNN
- 예: 문장, 음성
- 이전 시점으로 나온 (hidden state)를 현재 시점의 입력과 함께 처리

같은 가중치를 (하나의 시점)에서 공유함

RNN의 한계: 기울기 소실, 장기 의존성

RNN 들: RNN, LSTM, GRU

RNN 입출력 종류: 1:n, n:1, n:m

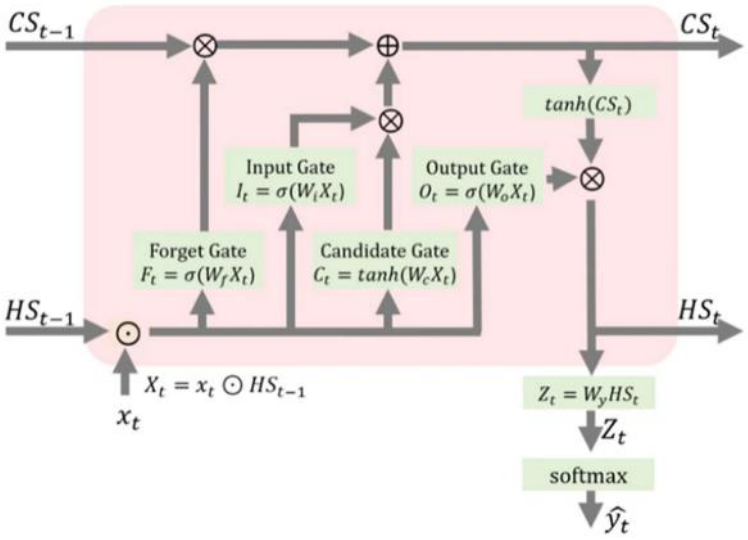
RNN의 기울기 소실, 장기 의존성 해결: LSTM
- 기존 (hidden state)에 (cell state)를 추가

필요한 정보는 (기억)
불필요한 정보 (약화)
새로운 정보 (선택적 저장)

장기기억 유지: Long-Term
단기 정보 처리: Short-term
기억: Memory

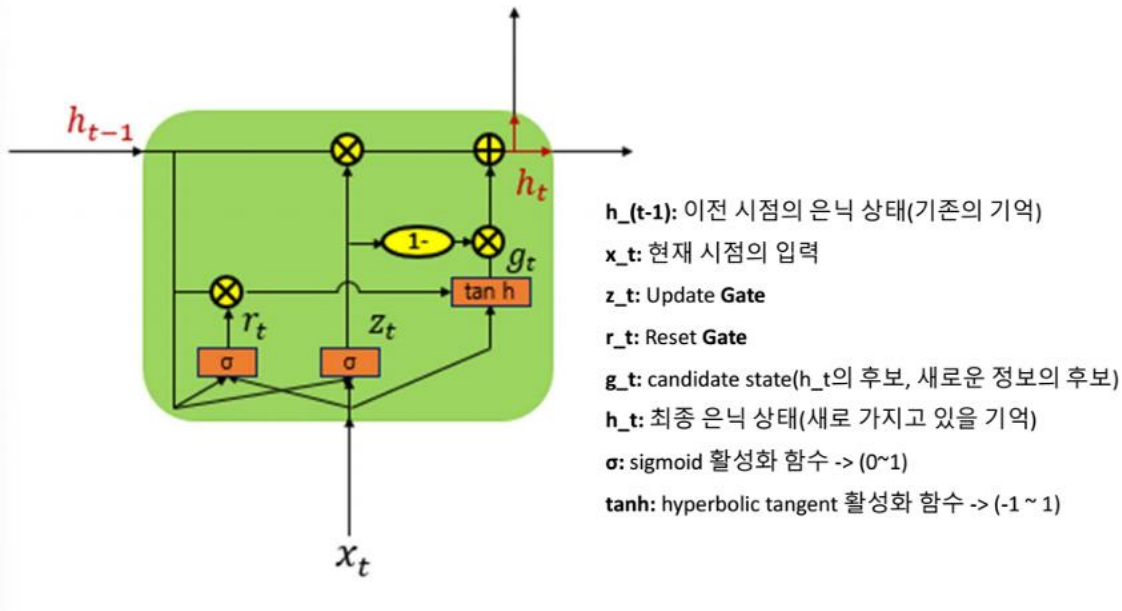
LSTM에서 장기 기억 저장 맡는 것: Cell State

어떤 정보 잊을지: Forget Gate
새로 들어올 정보의 값(초안 생성): Candidate Gate
어떤 정보를 새로 기억할지: Input Gate
현재 시점의 출력: Output Gate



이전 (hidden state)와 현재 입력 값을 받아서 concatenate하고
이전 (cell state)에서 잊을 것은 잊고(forget gate), 들어오는 입력 중 기억할 것을 골라 기억해서 (candidate gate, input gate) LSTM의 (cell state)를 업데이트 함
이후 출력할 것을 골라 출력(output gate)해서 LSTM의 (hidden state)를 업데이트 함

LSTM 성능유사, 경량화: GRU
- 게이트: Update Gate, Reset Gate
- GRU는 (Hidden State)만 사용함



3x3 합성곱을 지속적 수행: VGGNet

스킵 커넥션: ResNet
- 스킵 커넥션 다른 표현: 레지듀얼 커넥션

nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=stride, padding=1, bias=False)

```
def forward(self, x):
    residual = x

    out = self.conv1(x)
    out = self.relu(out)
    out = self.conv2(out)

    out += residual
    out = self.relu(out)

    return out
```

설명: skip connection 구현